PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PES UNIVERSITY
CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Exception Handling, File I/O, Modules and Packages

Pandas DataFrame Operations

Lekha A
Computer Applications

PROGRAMMING WITH PYTHON

Pandas

PES UNIVERSITY
CELEBRATING 50 YEARS

- Pandas is a popular Python package for data science.
- It offers powerful, expressive and flexible data structures that make data manipulation and analysis easy.

PROGRAMMING WITH PYTHON

DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- They are defined as two-dimensional labelled data structures with columns of potentially different types.
- Pandas DataFrame consists of three main components
  - Data
  - Index
  - Columns

PROGRAMMING WITH PYTHON
DataFrame - Creation

PES UNIVERSITY
CELEBRATING 50 YEARS

- A Pandas DataFrame can be created
  - Using Python Dictionary
  - Using Python List
  - From a File
  - Creating an Empty DataFrame

PROGRAMMING WITH PYTHON
DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- ### Creation of dataframe using dictionary

```
data = {'Name': ['Amora', 'Beast', 'Captain America'],
        'Age': [25, 30, 35],
        'City': ['New York', 'London', 'Paris']}
df = pd.DataFrame(data)
print(df)
```

```
              Name  Age      City
0            Amora   25  New York
1            Beast   30    London
2  Captain America   35     Paris
```

Creation of dataframe using lists

```
data = [['Deadpool', 25, 'New York'],
        ['Elektra', 30, 'London'],
        ['Falcon', 35, 'Paris']]
df = pd.DataFrame(data, columns=['Name', 'Age', 'City'])
print(df)
```

```
       Name  Age      City
0  Deadpool   25  New York
1   Elektra   30    London
2    Falcon   35     Paris
```

PES UNIVERSITY
CELEBRATING 50 YEARS

Creation of dataframe from a file

```
df = pd.read_csv('data.csv')
print(df)
```

```
       Name  Age      City
0    Gamora   25  New York
1   Hawkeye   30    London
2  Iron Man   20     Paris
```

```
df = pd.read_json('data.json')
print(df)
```

```
          Name  Age      City
0   Jean Grey   25  New York
1  Killmonger   30    London
2       Logan   20     Paris
```

Creation of an empty dataframe

```
df = pd.DataFrame()
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
```

PROGRAMMING WITH PYTHON

DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Inspect head, tail, shape and columns
  - The first few rows are viewed with df.head().
  - The last few rows are viewed with df.tail().
  - The total size is checked with df.shape.
  - Column names are checked with df.columns.
  - The index is checked with df.index.

PROGRAMMING WITH PYTHON

DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

Inspection

```
df.head()
```

	Name	Age	City
0	Jean Grey	25	New York
1	Killmonger	30	London
2	Logan	20	Paris

```
df.tail()
```

	Name	Age	City
0	Jean Grey	25	New York
1	Killmonger	30	London
2	Logan	20	Paris

```
df.shape
```

```
(3, 3)
```

```
df.columns
```

```
Index(['Name', 'Age', 'City'], dtype='object')
```

```
df.index
```

```
RangeIndex(start=0, stop=3, step=1)
```

PROGRAMMING WITH PYTHON
DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Column data types are checked with df.dtypes.
- One column is converted to a different type.

```
df.dtypes
```

```
Name     object
Age       int64
City     object
dtype: object
```

Convertion of columns

```
df["Age"] = df["Age"].astype("float64")
```

```
print(df)
```

```
          Name   Age       City
0   Jean Grey  25.0   New York
1  Killmonger  30.0     London
2       Logan  20.0      Paris
```

PROGRAMMING WITH PYTHON
DataFrame

- Select columns and rows with loc and iloc.
- Rows by label are selected using loc.

```
df.loc[0:2]
```

	Name	Age	City
0	Jean Grey	25.0	New York
1	Killmonger	30.0	London
2	Logan	20.0	Paris

Select columns and rows

```
df['Name']
```

```
0     Jean Grey
1    Killmonger
2         Logan
Name: Name, dtype: object
```

```
df[['Name','Age']]
```

	Name	Age
0	Jean Grey	25.0
1	Killmonger	30.0
2	Logan	20.0

PROGRAMMING WITH PYTHON

DataFrame

- Rows by integer position are selected using iloc.
- Row and column together are selected.

```
df.iloc[0:1]
```

	Name	Age	City
0	Jean Grey	25.0	New York

```
df.loc[0:2,['Name','Age']]
```

	Name	Age
0	Jean Grey	25.0
1	Killmonger	30.0
2	Logan	20.0

PROGRAMMING WITH PYTHON
DataFrame

- Check for missing values per column using df.isna.sum().
- Rows with missing values are checked using df.isna().any(axis=1).sum()
- notna() is tried to see non-missing positions.

Checking for missing values per column

```
df.isna().sum()
```

```
Product       0
 Qty          0
 Amount       1
 Region       0
 Customer     0
 City         0
Unnamed: 6    8
dtype: int64
```

Check for missing values per row

```
df.isna().any(axis=1).sum()
```

```
np.int64(8)
```

PROGRAMMING WITH PYTHON
DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Missing values in that column are filled with the mean or median.

Clean the column names

```
df.columns = df.columns.str.strip()
```

```
print(df.columns)
```

```
Index(['Product', 'Qty', 'Amount', 'Region', 'Customer', 'City', 'Unnamed: 6'], dtype='object')
```

Convert all values of Amount to strings and remove quotes and commas. Then keep only the digits.

```
df["Amount"] = (
df["Amount"]
.astype(str)
.str.replace('"', '', regex=False)
.str.replace(',', '', regex=False)
.str.extract(r'(\d+)', expand=False) # take only the numeric part
)
```

```
print(df["Amount"])
```

PROGRAMMING WITH PYTHON
DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Missing values in that column are filled with the mean or median.

Convert Amount to numeric

```
df["Amount"] = pd.to_numeric(df["Amount"], errors="coerce")
print(df.dtypes)
```

```
Product        object
Qty            object
Amount        float64
Region         object
Customer       object
City           object
Unnamed: 6     object
dtype: object
```

Fill missing values with the median

```
df["Amount"] = df["Amount"].fillna(df["Amount"].median())
```

PROGRAMMING WITH PYTHON
DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Drop a column

Drop a column

```
df = df.loc[:, ~df.columns.str.contains("^Unnamed")]
## df = df.drop(columns=["Unnamed: 6"])
## df = df.iloc[:, :-1] - drop by index position
## df = df.dropna(axis=1, how="all") Drop columns with all missing values
```

```
df
```

	Product	Qty	Amount	Region	Customer	City
0	Mobile	10	12000.0	south	Rahul	"Bengaluru"
1	Laptop		45000.0	North	SHREYA	Delhi
2	Headphones	5	1500.0	west	Arjun	Mumbai
3	Charger	3	1200.0	east	Meera	Kolkata
4	Camera	2	55000.0	South		Chennai
5	Tablet		22.0	500"	north	Rohan
6	Mouse	7	800.0	west	anita	Pune
7	Keyboard	4	1500.0	East	karthik	"Bhubaneswar"
8	Monitor		12.0	800"	south	
9	Speaker	6	3200.0	west	Vikram	Ahmedabad

PROGRAMMING WITH PYTHON
DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Standardize text columns

Remove quotes in all columns

```
df = df.apply(lambda col: col.astype(str).str.replace('"', '', regex=False))
```

```
df
```

	Product	Qty	Amount	Region	Customer	City
0	Mobile	10	12000.0	south	Rahul	Bengaluru
1	Laptop		45000.0	north	SHREYA	Delhi
2	Headphones	5	1500.0	west	Arjun	Mumbai
3	Charger	3	1200.0	east	Meera	Kolkata
4	Camera	2	55000.0	south		Chennai
5	Tablet		22.0	500	north	Rohan
6	Mouse	7	800.0	west	anita	Pune
7	Keyboard	4	1500.0	east	karthik	Bhubaneswar
8	Monitor		12.0	800	south	
9	Speaker	6	3200.0	west	Vikram	Ahmedabad

PROGRAMMING WITH PYTHON

DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Rename columns for clarity
- Column names are printed with df.columns.
- Can create a rename dictionary.
  - Example: rename_map = {"Prod": "Product", "Qty": "Quantity"}
- df = df.rename(columns=rename_map) is applied
- Columns are checked again to confirm clean names.

PROGRAMMING WITH PYTHON
DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Rename columns for clarity
  - Column names are printed with df.columns.
  - Can create a renamed dictionary.
    - Example: rename_map = {"Prod": "Product", "Qty": "Quantity"}
  - df = df.rename(columns=rename_map) is applied

PROGRAMMING WITH PYTHON
DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Convert numeric text columns to proper numbers
  - A column that looks numeric but is stored as object is identified.
    - Example: df["Amount"].dtype is checked.
  - Non-numeric characters are removed if needed.
    - Example: df["Amount"] = df["Amount"].str.replace(",", "")

PROGRAMMING WITH PYTHON

DataFrame

- Convert numeric text columns to proper numbers
  - The column is converted to float or int.
    - Example: df["Amount"] = df["Amount"].astype("float64")
- dtypes is checked again to confirm numeric type.

PROGRAMMING WITH PYTHON

DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

-
```
df["Amount"] = df["Amount"].str.replace(",", "")
```

```
df["Amount"] = df["Amount"].astype("float64")
```

```
print(df.dtypes)
```

```
Product      object
Qty          object
Amount      float64
Region       object
Customer     object
City         object
dtype: object
```

PROGRAMMING WITH PYTHON
DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Filter the data

Filter the data

```
high_sales = df["Amount"] > 2000
```

```
high_sales
```

```
0     True
1     True
2    False
3    False
4     True
5    False
6    False
7    False
8    False
9     True
Name: Amount, dtype: bool
```

Create the filtered DataFrame

```
df_high = df[high_sales]
```

```
len(df_high)
```

```
4
```

```
df_high
```

	Product	Qty	Amount	Region	Customer	City
0	Mobile	10	12000.0	south	Rahul	Bengaluru
1	Laptop		45000.0	north	SHREYA	Delhi
4	Camera	2	55000.0	south		Chennai
9	Speaker	6	3200.0	west	Vikram	Ahmedabad

PROGRAMMING WITH PYTHON

DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Combine multiple conditions.
- They are combined with & and |.

Combine multiple conditions

```
cond_amount = df["Amount"] > 2000
cond_region = df["Region"] == "south"
```

```
df_target = df[cond_amount & cond_region]
```

```
df_target
```

	Product	Qty	Amount	Region	Customer	City
0	Mobile	10	12000.0	south	Rahul	Bengaluru
4	Camera	2	55000.0	south		Chennai

```
df_target1 = df[cond_amount | cond_region]
```

```
df_target1
```

	Product	Qty	Amount	Region	Customer	City
0	Mobile	10	12000.0	south	Rahul	Bengaluru
1	Laptop		45000.0	north	SHREYA	Delhi
4	Camera	2	55000.0	south		Chennai
9	Speaker	6	3200.0	west	Vikram	Ahmedabad

PROGRAMMING WITH PYTHON
DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Sort values by one or more columns.
  - Example: The DataFrame is sorted by "Amount".
    - df_sorted = df.sort_values("Amount", ascending=False)
- Sorting by multiple columns is also possible.
  - df_sorted2 = df.sort_values(["Region", "Amount"])

PROGRAMMING WITH PYTHON

Data Frame

- ## Sorting of Values

```
df_sorted = df.sort_values("Amount", ascending=False)
```

```
df_sorted
```

	Product	Qty	Amount	Region	Customer	City
4	Camera	2	55000.0	south		Chennai
1	Laptop		45000.0	north	SHREYA	Delhi
0	Mobile	10	12000.0	south	Rahul	Bengaluru
9	Speaker	6	3200.0	west	Vikram	Ahmedabad
7	Keyboard	4	1500.0	east	karthik	Bhubaneswar
2	Headphones	5	1500.0	west	Arjun	Mumbai
3	Charger	3	1200.0	east	Meera	Kolkata
6	Mouse	7	800.0	west	anita	Pune
5	Tablet		22.0	500	north	Rohan
8	Monitor		12.0	800	south	

```
df_sorted2 = df.sort_values(["Region", "Amount"])
```

```
df_sorted2
```

	Product	Qty	Amount	Region	Customer	City
5	Tablet		22.0	500	north	Rohan
8	Monitor		12.0	800	south	
3	Charger	3	1200.0	east	Meera	Kolkata
7	Keyboard	4	1500.0	east	karthik	Bhubaneswar
1	Laptop		45000.0	north	SHREYA	Delhi
0	Mobile	10	12000.0	south	Rahul	Bengaluru
4	Camera	2	55000.0	south		Chennai
6	Mouse	7	800.0	west	anita	Pune
2	Headphones	5	1500.0	west	Arjun	Mumbai
9	Speaker	6	3200.0	west	Vikram	Ahmedabad

PROGRAMMING WITH PYTHON

Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Create derived columns using arithmetic.
  - Example: A new column "Tax" is created as 10 percent of Amount.
    - df["Tax"] = df["Amount"] * 0.10
  - A "TotalWithTax" column is created.
    - df["TotalWithTax"] = df["Amount"] + df["Tax"]

PROGRAMMING WITH PYTHON
Data Frame

- Creating Derived Columns

Create Derived Columns using Arithmetic

```
df["Tax"] = df["Amount"] * 0.10
```

```
df.dtypes
```

```
Product      object
Qty          object
Amount      float64
Region       object
Customer     object
City         object
Tax         float64
dtype: object
```

```
df
```

	Product	Qty	Amount	Region	Customer	City	Tax
0	Mobile	10	12000.0	south	Rahul	Bengaluru	1200.0
1	Laptop		45000.0	north	SHREYA	Delhi	4500.0
2	Headphones	5	1500.0	west	Arjun	Mumbai	150.0
3	Charger	3	1200.0	east	Meera	Kolkata	120.0
4	Camera	2	55000.0	south		Chennai	5500.0
5	Tablet		22.0	500	north	Rohan	2.2
6	Mouse	7	800.0	west	anita	Pune	80.0
7	Keyboard	4	1500.0	east	karthik	Bhubaneswar	150.0
8	Monitor		12.0	800	south		1.2
9	Speaker	6	3200.0	west	Vikram	Ahmedabad	320.0

PROGRAMMING WITH PYTHON
Data Frame

- Creating Derived Columns

```
df["TotalWithTax"] = df["Amount"] + df["Tax"]
```

```
df
```

	Product	Qty	Amount	Region	Customer	City	Tax	TotalWithTax
0	Mobile	10	12000.0	south	Rahul	Bengaluru	1200.0	13200.0
1	Laptop		45000.0	north	SHREYA	Delhi	4500.0	49500.0
2	Headphones	5	1500.0	west	Arjun	Mumbai	150.0	1650.0
3	Charger	3	1200.0	east	Meera	Kolkata	120.0	1320.0
4	Camera	2	55000.0	south		Chennai	5500.0	60500.0
5	Tablet		22.0	500	north	Rohan	2.2	24.2
6	Mouse	7	800.0	west	anita	Pune	80.0	880.0
7	Keyboard	4	1500.0	east	karthik	Bhubaneswar	150.0	1650.0
8	Monitor		12.0	800	south		1.2	13.2
9	Speaker	6	3200.0	west	Vikram	Ahmedabad	320.0	3520.0

PROGRAMMING WITH PYTHON
Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Profile columns with value_counts and describe.
  - A categorical column is analyzed.
    - Example: df["Region"].value_counts()
  - A numeric column is summarized.
    - Example: df["Amount"].describe()
- The information is read to understand range, median and spread.

PROGRAMMING WITH PYTHON
Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Profile columns with value_counts and describe

Categorical column is analyzed

```
df["Region"].value_counts()
```

```
Region
west     3
south    2
east     2
north    1
500      1
800      1
Name: count, dtype: int64
```

Numeric column is summarized

```
df["Amount"].describe()
```

```
count       10.000000
mean     12023.400000
std      20453.739545
min         12.000000
25%        900.000000
50%       1500.000000
75%       9800.000000
max      55000.000000
Name: Amount, dtype: float64
```

PROGRAMMING WITH PYTHON
Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Grouping and Aggregating
- Group by a single categorical column
  - Example: Sales are grouped by "Region".
    - grp_region = df.groupby("Region")
  - The total Amount per region is computed.
    - sales_by_region = grp_region["Amount"].sum()

PROGRAMMING WITH PYTHON
Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- View the group names
  - grp_region.groups.keys()
- View the row indexes belonging to each group
  - grp_region.groups
- View one specific group
  - grp_region.get_group("south")

PROGRAMMING WITH PYTHON
Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

Group by a single categorical column

```
grp_region = df.groupby("Region")
```

```
type(grp_region)
```

```
pandas.core.groupby.generic.DataFrameGroupBy
```

View the group names

```
grp_region.groups.keys()
```

```
dict_keys(['500', '800', 'east', 'north', 'south', 'west'])
```

View the row indexes belonging to each group

```
grp_region.groups
```

```
{'500': [5], '800': [8], 'east': [3, 7], 'north': [1], 'south': [0, 4], 'west': [2, 6, 9]}
```

View one specific group

```
grp_region.get_group("south")
```

	Product	Qty	Amount	Region	Customer	City	Tax	TotalWithTax
0	Mobile	10	12000.0	south	Rahul	Bengaluru	1200.0	13200.0
4	Camera	2	55000.0	south		Chennai	5500.0	60500.0

PROGRAMMING WITH PYTHON

Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

Loop through groups (for inspection only)

-

```
for name, group in grp_region:
    print("Group:", name)
    print(group)
```

```
Group: 500
  Product Qty  Amount Region Customer   City  Tax  TotalWithTax
5  Tablet        22.0    500    north  Rohan  2.2          24.2
Group: 800
   Product Qty  Amount Region Customer City  Tax  TotalWithTax
8  Monitor        12.0    800    south       1.2          13.2
Group: east
    Product Qty  Amount Region   Customer          City    Tax  TotalWithTax
3   Charger   3  1200.0   east      Meera       Kolkata  120.0        1320.0
7  Keyboard   4  1500.0   east   karthik    Bhubaneswar  150.0        1650.0
Group: north
  Product Qty   Amount Region  Customer    City     Tax  TotalWithTax
1  Laptop      45000.0  north   SHREYA    Delhi  4500.0       49500.0
Group: south
   Product Qty   Amount Region Customer        City     Tax  TotalWithTax
0  Mobile   10  12000.0  south    Rahul   Bengaluru  1200.0       13200.0
4  Camera    2  55000.0  south             Chennai  5500.0       60500.0
Group: west
      Product Qty  Amount Region Customer        City    Tax  TotalWithTax
2  Headphones   5  1500.0   west   Arjun       Mumbai  150.0        1650.0
6       Mouse   7   800.0   west   anita         Pune   80.0         880.0
9     Speaker   6  3200.0   west   Vikram   Ahmedabad  320.0        3520.0
```

PROGRAMMING WITH PYTHON

Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- ## Aggregation

```
sales_by_region = grp_region["Amount"].sum()
```

```
sales_by_region
```

```
Region
500          22.0
800          12.0
east       2700.0
north     45000.0
south     67000.0
west       5500.0
Name: Amount, dtype: float64
```

Data Frame

- Compute multiple aggregates for one column.
  - Example: Both mean and sum of Amount per Region are computed.
    - region_summary = grp_region["Amount"].agg(["sum", "mean", "count"])

PROGRAMMING WITH PYTHON

Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Compute multiple aggregates for one column.

Compute multiple aggregates for one column

```
region_summary = grp_region["Amount"].agg(["sum", "mean", "count"])
```

```
region_summary
```

Region	sum	mean	count
500	22.0	22.000000	1
800	12.0	12.000000	1
east	2700.0	1350.000000	2
north	45000.0	45000.000000	1
south	67000.0	33500.000000	2
west	5500.0	1833.333333	3

PROGRAMMING WITH PYTHON
Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Group by multiple columns
  - Example: Grouping by "Region" and "Product" together is done.
    - grp_rp = df.groupby(["Region", "Product"])

PROGRAMMING WITH PYTHON

Data Frame

- Group by multiple columns

Aggregate

```
rp_summary = grp_rp["Amount"].sum()
```

```
rp_summary
```

```
Region  Product
500     Tablet          22.0
800     Monitor         12.0
east    Charger       1200.0
        Keyboard      1500.0
north   Laptop       45000.0
south   Camera       55000.0
        Mobile       12000.0
west    Headphones    1500.0
        Mouse          800.0
        Speaker       3200.0
Name: Amount, dtype: float64
```

```
grp_rp = df.groupby(["Region", "Product"])
```

```
for name, group in grp_rp:
    print("Group:", name)
    print(group)
```

```
Group: ('500', 'Tablet')
  Product Qty  Amount Region Customer   City  Tax  TotalWithTax
5  Tablet        22.0    500    north  Rohan  2.2          24.2
Group: ('800', 'Monitor')
   Product Qty  Amount Region Customer City  Tax  TotalWithTax
8  Monitor        12.0    800    south       1.2          13.2
Group: ('east', 'Charger')
   Product Qty  Amount Region Customer     City    Tax  TotalWithTax
3  Charger   3  1200.0   east    Meera  Kolkata  120.0        1320.0
Group: ('east', 'Keyboard')
    Product Qty  Amount Region   Customer          City    Tax  TotalWithTax
7  Keyboard   4  1500.0   east   karthik    Bhubaneswar  150.0        1650.0
Group: ('north', 'Laptop')
  Product Qty   Amount Region  Customer    City     Tax  TotalWithTax
1  Laptop      45000.0  north   SHREYA    Delhi  4500.0       49500.0
Group: ('south', 'Camera ')
   Product Qty   Amount Region Customer     City     Tax  TotalWithTax
4  Camera    2  55000.0  south           Chennai  5500.0       60500.0
Group: ('south', 'Mobile ')
   Product Qty   Amount Region Customer        City     Tax  TotalWithTax
0  Mobile   10  12000.0  south    Rahul   Bengaluru  1200.0       13200.0
Group: ('west', 'Headphones')
      Product Qty  Amount Region Customer     City    Tax  TotalWithTax
2  Headphones   5  1500.0   west   Arjun    Mumbai  150.0        1650.0
Group: ('west', 'Mouse')
  Product Qty  Amount Region Customer  City   Tax  TotalWithTax
6   Mouse   7   800.0   west   anita   Pune  80.0         880.0
Group: ('west', 'Speaker')
   Product Qty  Amount Region Customer        City    Tax  TotalWithTax
9  Speaker   6  3200.0   west   Vikram   Ahmedabad  320.0        3520.0
```

PROGRAMMING WITH PYTHON

Data Frame

- Custom aggregation

Custom Aggregation

```
summary = df.groupby("Region").agg(
Total_Amount=("Amount", "sum"),
Avg_Amount=("Amount", "mean"),
Total_Tax=("Tax", "sum"),
Avg_Tax=("Tax", "mean"),
Total_Quantity=("Qty", "sum"),
Unique_Customers=("Customer", "nunique")
)
```

```
summary
```

Region	Total_Amount	Avg_Amount	Total_Tax	Avg_Tax	Total_Quantity	Unique_Customers
500	22.0	22.000000	2.2	2.200000		1
800	12.0	12.000000	1.2	1.200000		1
east	2700.0	1350.000000	270.0	135.000000	34	2
north	45000.0	45000.000000	4500.0	4500.000000		1
south	67000.0	33500.000000	6700.0	3350.000000	102	2
west	5500.0	1833.333333	550.0	183.333333	576	3

PROGRAMMING WITH PYTHON

Data Frame

- Custom aggregation

Custom Aggregation

```
summary = df.groupby("Region").agg(
Total_Amount=("Amount", "sum"),
Avg_Amount=("Amount", "mean"),
Total_Tax=("Tax", "sum"),
Avg_Tax=("Tax", "mean"),
Total_Quantity=("Qty", "sum"),
Unique_Customers=("Customer", "nunique")
)
```

```
summary
```

Region	Total_Amount	Avg_Amount	Total_Tax	Avg_Tax	Total_Quantity	Unique_Customers
500	22.0	22.000000	2.2	2.200000		1
800	12.0	12.000000	1.2	1.200000		1
east	2700.0	1350.000000	270.0	135.000000	34	2
north	45000.0	45000.000000	4500.0	4500.000000		1
south	67000.0	33500.000000	6700.0	3350.000000	102	2
west	5500.0	1833.333333	550.0	183.333333	576	3

PROGRAMMING WITH PYTHON

Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Reshape grouped data with unstack.
  - Example: The Region–Product summary is used.
    - rp_table = rp_summary.unstack("Product")
- The result looks like a pivot table with Regions as rows and Products as columns.
  - fillna(0) is applied if missing combinations exist.

Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

Reshape grouped data

```
rp_table = rp_summary.unstack("Product")
```

```
rp_table
```

Product	Camera	Charger	Headphones	Keyboard	Laptop	Mobile	Monitor	Mouse	Speaker	Tablet
Region										
500	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	22.0
800	NaN	NaN	NaN	NaN	NaN	NaN	12.0	NaN	NaN	NaN
east	NaN	1200.0	NaN	1500.0	NaN	NaN	NaN	NaN	NaN	NaN
north	NaN	NaN	NaN	NaN	45000.0	NaN	NaN	NaN	NaN	NaN
south	55000.0	NaN	NaN	NaN	NaN	12000.0	NaN	NaN	NaN	NaN
west	NaN	NaN	1500.0	NaN	NaN	NaN	NaN	800.0	3200.0	NaN

PROGRAMMING WITH PYTHON
Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Combine two data frames

```
sales = pd.read_csv("sales.csv")
products = pd.read_csv("products.csv")
```

```
sales.columns
```

```
Index(['OrderID', 'ProductID', 'Qty', 'Amount', 'Region', 'Customer', 'City',
       'Date'],
      dtype='object')
```

```
products.columns
```

```
Index(['ProductID', 'ProductName', 'Category', 'Brand'], dtype='object')
```

PROGRAMMING WITH PYTHON

Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- It shows that ProductID is common to both the data frames.
- Uniqueness of ProductID in products is checked with
  - products["ProductID"].nunique().
- Count is compared to len(products) to confirm a proper lookup table.

Check for uniqueness

```
products["ProductID"].nunique()
```

```
8
```

```
len(products)
```

```
8
```

PROGRAMMING WITH PYTHON
Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Merge using inner join
  - merged_inner = pd.merge(sales, products, on="ProductID", how="inner")
- It Keep only rows where ProductID exists in both sales and products.
- If a sales row has a ProductID not present in products, it is removed.
- Useful when there is a need for only “valid” or “matched” transactions.

PROGRAMMING WITH PYTHON

Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Check number of rows
  - print(len(sales), len(merged_inner))
- If merged_inner is smaller, some sales records did not have a matching ProductID.

PROGRAMMING WITH PYTHON

Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- ## Merge using inner join

```
merged_inner = pd.merge(sales, products, on="ProductID", how="inner")
```

```
merged_inner
```

	OrderID	ProductID	Qty	Amount	Region	Customer	City	Date	ProductName	Category	Brand
0	1001	P101	2	24000	south	Rahul	Bengaluru	2024-01-12	Mobile	Electronics	Samsung
1	1002	P103	1	1500	west	Arjun	Mumbai	2024-01-13	Headphones	Accessories	Boat
2	1003	P102	3	45000	north	Shreya	Delhi	2024-01-14	Laptop	Electronics	HP
3	1004	P104	1	1200	east	Meera	Kolkata	2024-01-15	Charger	Accessories	MI
4	1005	P105	1	55000	south	Anand	Chennai	2024-01-15	Camera	Electronics	Sony
5	1006	P101	1	12000	west	Anita	Pune	2024-01-16	Mobile	Electronics	Samsung
6	1007	P106	2	1500	east	Karthik	Bhubaneswar	2024-01-16	Keyboard	Accessories	Logitech
7	1008	P107	1	12800	south	Dev	Hyderabad	2024-01-17	Monitor	Electronics	Dell
8	1009	P108	3	3200	west	Vikram	Ahmedabad	2024-01-17	Speaker	Accessories	JBL
9	1010	P102	1	45000	north	Rohan	New Delhi	2024-01-18	Laptop	Electronics	HP

```
print(len(sales), len(merged_inner))
```

```
10 10
```

PROGRAMMING WITH PYTHON

Data Frame

PES UNIVERSITY
CELEBRATING 50 YEARS

- ## Merge using inner join

```
merged_inner = pd.merge(sales, products, on="ProductID", how="inner")
```

```
merged_inner
```

	OrderID	ProductID	Qty	Amount	Region	Customer	City	Date	ProductName	Category	Brand
0	1001	P101	2	24000	south	Rahul	Bengaluru	2024-01-12	Mobile	Electronics	Samsung
1	1002	P103	1	1500	west	Arjun	Mumbai	2024-01-13	Headphones	Accessories	Boat
2	1003	P102	3	45000	north	Shreya	Delhi	2024-01-14	Laptop	Electronics	HP
3	1004	P104	1	1200	east	Meera	Kolkata	2024-01-15	Charger	Accessories	MI
4	1005	P105	1	55000	south	Anand	Chennai	2024-01-15	Camera	Electronics	Sony
5	1006	P101	1	12000	west	Anita	Pune	2024-01-16	Mobile	Electronics	Samsung
6	1007	P106	2	1500	east	Karthik	Bhubaneswar	2024-01-16	Keyboard	Accessories	Logitech
7	1008	P107	1	12800	south	Dev	Hyderabad	2024-01-17	Monitor	Electronics	Dell
8	1009	P108	3	3200	west	Vikram	Ahmedabad	2024-01-17	Speaker	Accessories	JBL
9	1010	P102	1	45000	north	Rohan	New Delhi	2024-01-18	Laptop	Electronics	HP

```
print(len(sales), len(merged_inner))
```

```
10 10
```

PROGRAMMING WITH PYTHON
Data Frames

PES UNIVERSITY
CELEBRATING 50 YEARS

- Merge using left join
  - merged_left = pd.merge(sales, products, on="ProductID", how="left")
    - Keep every row from sales (left table).
    - Add product details wherever ProductID matches.
    - If a ProductID is missing in products, those product fields turn into NaN.
- Check number of rows
  - print(len(sales), len(merged_left))

PROGRAMMING WITH PYTHON
Data Frames

PES UNIVERSITY
CELEBRATING 50 YEARS

- ## Merge using left join

```
merged_left = pd.merge(sales, products, on="ProductID", how="left")
```

```
print(len(sales), len(merged_left))
```

```
10 10
```

```
merged_left
```

	OrderID	ProductID	Qty	Amount	Region	Customer	City	Date	ProductName	Category	Brand
0	1001	P101	2	24000	south	Rahul	Bengaluru	2024-01-12	Mobile	Electronics	Samsung
1	1002	P103	1	1500	west	Arjun	Mumbai	2024-01-13	Headphones	Accessories	Boat
2	1003	P102	3	45000	north	Shreya	Delhi	2024-01-14	Laptop	Electronics	HP
3	1004	P104	1	1200	east	Meera	Kolkata	2024-01-15	Charger	Accessories	MI
4	1005	P105	1	55000	south	Anand	Chennai	2024-01-15	Camera	Electronics	Sony
5	1006	P101	1	12000	west	Anita	Pune	2024-01-16	Mobile	Electronics	Samsung
6	1007	P106	2	1500	east	Karthik	Bhubaneswar	2024-01-16	Keyboard	Accessories	Logitech
7	1008	P107	1	12800	south	Dev	Hyderabad	2024-01-17	Monitor	Electronics	Dell
8	1009	P108	3	3200	west	Vikram	Ahmedabad	2024-01-17	Speaker	Accessories	JBL
9	1010	P102	1	45000	north	Rohan	New Delhi	2024-01-18	Laptop	Electronics	HP

PROGRAMMING WITH PYTHON
Data Frames

PES UNIVERSITY
CELEBRATING 50 YEARS

- Load the second sales file
  - sales_q2 = pd.read_csv("sales_q2.csv")
- Check that both DataFrames have the same structure
  - print(sales.columns)
  - print(sales_q2.columns)

Data Frames

- Both files must use identical column names and order.
  - If anything is different, concatenation will misalign columns.
- Vertically concatenate the two DataFrames
  - sales_all = pd.concat([sales, sales_q2], ignore_index=True)

PES UNIVERSITY
CELEBRATING 50 YEARS

- ## Concatenate DataFrames vertically

```
sales_q2 = pd.read_csv("sales_q2.csv")
```

Check that both DataFrames have the same structure

```
print(sales.columns)
print(sales_q2.columns)
```

```
Index(['OrderID', 'ProductID', 'Qty', 'Amount', 'Region', 'Customer', 'City',
       'Date'],
      dtype='object')
Index(['OrderID', 'ProductID', 'Qty', 'Amount', 'Region', 'Customer', 'City',
       'Date'],
      dtype='object')
```

PROGRAMMING WITH PYTHON

Data Frames

-

Vertically concatenate the two DataFrames

```
sales_all = pd.concat([sales, sales_q2], ignore_index=True)
```

```
sales_all
```

	OrderID	ProductID	Qty	Amount	Region	Customer	City	Date
0	1001	P101	2	24000	south	Rahul	Bengaluru	2024-01-12
1	1002	P103	1	1500	west	Arjun	Mumbai	2024-01-13
2	1003	P102	3	45000	north	Shreya	Delhi	2024-01-14
3	1004	P104	1	1200	east	Meera	Kolkata	2024-01-15
4	1005	P105	1	55000	south	Anand	Chennai	2024-01-15
5	1006	P101	1	12000	west	Anita	Pune	2024-01-16
6	1007	P106	2	1500	east	Karthik	Bhubaneswar	2024-01-16
7	1008	P107	1	12800	south	Dev	Hyderabad	2024-01-17
8	1009	P108	3	3200	west	Vikram	Ahmedabad	2024-01-17
9	1010	P102	1	45000	north	Rohan	New Delhi	2024-01-18
10	2001	P101	1	12000	south	Sneha	Bengaluru	2024-04-05
11	2002	P102	2	90000	north	Amit	Delhi	2024-04-06
12	2003	P103	3	4500	west	Raj	Mumbai	2024-04-08
13	2004	P104	2	2400	east	Meera	Kolkata	2024-04-10
14	2005	P105	1	53000	south	Vinay	Chennai	2024-05-01
15	2006	P106	4	3200	west	Anjali	Pune	2024-05-03
16	2007	P107	2	25600	south	Kiran	Hyderabad	2024-05-15
17	2008	P108	5	5400	west	Neha	Ahmedabad	2024-05-20
18	2009	P101	3	36000	north	Varun	Jaipur	2024-06-02
19	2010	P102	1	45000	east	Diya	Bhubaneswar	2024-06-10

PROGRAMMING WITH PYTHON

Data Frames

PES UNIVERSITY
CELEBRATING 50 YEARS

- See a combined row count.

```
print(sales.shape, sales_q2.shape, sales_all.shape)
```

```
(10, 8) (10, 8) (20, 8)
```

PROGRAMMING WITH PYTHON

Data Frames

PES UNIVERSITY
CELEBRATING 50 YEARS

- Drop unwanted columns

Drop Unwanted Columns

```
cols_needed = ["OrderID", "ProductID", "ProductName", "Region", "Amount", "Date"]
final_df = merged_left[cols_needed]
```

```
final_df
```

	OrderID	ProductID	ProductName	Region	Amount	Date
0	1001	P101	Mobile	south	24000	2024-01-12
1	1002	P103	Headphones	west	1500	2024-01-13
2	1003	P102	Laptop	north	45000	2024-01-14
3	1004	P104	Charger	east	1200	2024-01-15
4	1005	P105	Camera	south	55000	2024-01-15
5	1006	P101	Mobile	west	12000	2024-01-16
6	1007	P106	Keyboard	east	1500	2024-01-16
7	1008	P107	Monitor	south	12800	2024-01-17
8	1009	P108	Speaker	west	3200	2024-01-17
9	1010	P102	Laptop	north	45000	2024-01-18

PROGRAMMING WITH PYTHON

DataFrame

PES UNIVERSITY
CELEBRATING 50 YEARS

- Export the DataFrame
  - final_df.to_csv("sales_final.csv", index=False)
  - final_df.to_excel("sales_final.xlsx", index=False)

sales_final.csv	7 seconds ago	460 B
sales_final.xlsx	5 seconds ago	5.3 KB

PROGRAMMING WITH PYTHON

Recap

PES UNIVERSITY
CELEBRATING 50 YEARS

- Pandas
- DataFrame
- Creation
- Storing

THANK YOU

Lekha A

Computer Applications